Colby Crutcher

Lab 10

Secure Coding

Task 1

1. cppcheck

```
colby@Colby-PC:/mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab10$ cppcheck --enable=all --inconclusive task1.c
Checking task1.c ...
task1.c:3:0: information: Include file: <stdio.h> not found. Please note: Cppcheck does not need standard library headers to get proper results. [missingIncludeSyste
m]
#include <stdio.h>
^
task1.c:4:0: information: Include file: <stdlib.h> not found. Please note: Cppcheck does not need standard library headers to get proper results. [missingIncludeSyst
em]
#include <stdlib.h>
^
task1.c:5:0: information: Include file: <string.h> not found. Please note: Cppcheck does not need standard library headers to get proper results. [missingIncludeSyst
em]
#include <string.h>
^
task1.c:54:8: portability: Assigning a pointer to an integer is not portable. [AssignmentAddressToInteger]
  *ptr = malloc(sizeof(int));
       ^
task1.c:30:41: error: Array 'arr[3]' accessed at index 5, which is out of bounds. [arrayIndexOutOfBounds]
  printf("Out-of-bounds read: %d\n", arr[5]);
                                        ^
task1.c:49:10: error: Buffer is accessed out of bounds: mem_buffer [bufferAccessOutOfBounds]
  strcpy(mem_buffer, "Welcome, how many errors are there?");
         ^
task1.c:34:3: warning: Obsolete function 'gets' called. It is recommended to use 'fgets' or 'gets_s' instead. [getsCalled]
  gets(buffer);
  ^
task1.c:45:3: error: Memory pointed to by 'buf2R1' is freed twice. [doubleFree]
  free(buf2R1);
  ^
task1.c:28:3: note: Memory pointed to by 'buf2R1' is freed twice.
  free(buf2R1);
  ^
task1.c:45:3: note: Memory pointed to by 'buf2R1' is freed twice.
  free(buf2R1);
  ^
task1.c:52:28: error: Null pointer dereference: ptr [nullPointer]
  printf("Pointer: %d\n", *ptr);
                           ^
task1.c:32:14: note: Assignment 'ptr=NULL', assigned value is 0
  int *ptr = NULL;
             ^
task1.c:52:28: note: Null pointer dereference
  printf("Pointer: %d\n", *ptr);
                           ^
task1.c:54:4: error: Null pointer dereference: ptr [nullPointer]
  *ptr = malloc(sizeof(int));
   ^
task1.c:32:14: note: Assignment 'ptr=NULL', assigned value is 0
  int *ptr = NULL;
             ^
task1.c:54:4: note: Null pointer dereference
  *ptr = malloc(sizeof(int));
   ^
task1.c:55:4: error: Null pointer dereference: ptr [nullPointer]
  *ptr = 10;
```

- Line 30 - arr[5] is accessed, and since arr was declared as int arr[3] (which only has indices of 0, 1, and 2), accessing index 5 points to memory outside the array's allocated boundaries.
- Line 49 - The 36-byte string (including the null terminator) exceeds the 10-byte allocation of mem_buffer,
- Line 45 - Frees the memory buf2R1 twice, which was already freed on line 28, which can cause corrupt memory, and can crash the program, or be exploited by an attacker to execute arbitrary code.
- Line 52 - Null pointer reference, which can cause the program to crash with a segmentation fault.

- Line 54 - code attempts to write to the ptr, but this is an invalid operation and will cause a crash.
- Line 55 - same as 54, attempts to write int 10 to the null memory address.

2. Flaw Finder

```
FINAL RESULTS:

task1.c:34:  [5] (buffer) gets:
  Does not check for buffer overflows (CWE-120, CWE-20). Use fgets() instead.
  gets(buffer);
task1.c:22:  [4] (format) printf:
  If format strings can be influenced by an attacker, they can be exploited
  (CWE-134). Use a constant for the format specification.
  printf(buffer);
task1.c:15:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.
  char buffer[64];
task1.c:48:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.
  char mem_buffer[10];
task1.c:49:  [2] (buffer) strcpy:
  Does not check for buffer overflows when copying to destination [MS-banned]
  (CWE-120). Consider using snprintf, strcpy_s, or strlcpy (warning: strncpy
  easily misused). Risk is low because the source is a constant string.
  strcpy(mem_buffer, "Welcome, how many errors are there?");
task1.c:43:  [1] (buffer) strncpy:
  Easily used incorrectly; doesn't always \0-terminate or check for invalid
  pointers [MS-banned] (CWE-120).
  strncpy(buf1R2, argv[1], BUFSIZE1-1);

ANALYSIS SUMMARY
```

- On line 34, it uses gets, which is a depricated C function, so it can be exploited using a buffer overflow. FlawFinder tells us to use fgets() instead, so we can do a bounds check, preventing a potential overflow.
- On line 22, we have a format string vulnerability because the buffer variable is populated by user input (argv[1]), passing it directly to printf without a format specifier, and this allows an attacker to input format tokens like %x or %n to read from or write to the program's memory.
- On line 15, and 48, we have a similar issue between the two, a statically-sized array. This is due to the char buffer[64] and char mem_buffer[10]. They arent vulnerabilites, but since they aren't managed later in the code,

they can fall victim of overflow attacks if the bounds aren't managed.

- On line 49, there is a strcpy vuln, which copies characters until it encounters a null terminator, not verifying size, which can easily lead to a buffer overflow
- Line 43 uses strncpy which is safer than strcpy, but can be misused. If the string is greater or equal to the buffer size, it can lead to an out-of-bound read.

3. Splint

```
colby@Colby-PC:/mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab10$ splint  task1.c
Splint 3.1.2 --- 21 Feb 2021

task1.c: (in function main)
task1.c:19:9: Return value (type int) ignored: snprintf(buffer,...
  Result returned by function call is not used. If this is intended, can cast
  result to (void) to eliminate message. (Use -retvalint to inhibit warning)
task1.c:22:3: Format string parameter to printf is not a compile-time constant:
                buffer
  Format parameter is not known at compile-time.  This can lead to security
  vulnerabilities because the arguments cannot be type checked. (Use
  -formatconst to inhibit warning)
task1.c:34:3: Use of gets leads to a buffer overflow vulnerability.  Use fgets
                instead: gets
  Use of function that may lead to buffer overflow. (Use -bufferoverflowhigh to
  inhibit warning)
task1.c:34:3: Return value (type char *) ignored: gets(buffer)
  Result returned by function call is not used. If this is intended, can cast
  result to (void) to eliminate message. (Use -retvalother to inhibit warning)
task1.c:43:11: Possibly null storage buf1R2 passed as non-null param:
                 strncpy (buf1R2, ...)
  A possibly null pointer is passed as a parameter corresponding to a formal
  parameter with no /*@null@*/ annotation.  If NULL may be used for this
  parameter, add a /*@null@*/ annotation to the function parameter declaration.
  (Use -nullpass to inhibit warning)
   task1.c:42:12: Storage buf1R2 may become null
task1.c:45:8: Dead storage buf2R1 passed as out parameter to free: buf2R1
  Memory is used after it has been released (either by passing as an only param
  or assigning to an only global). (Use -usereleased to inhibit warning)
   task1.c:28:8: Storage buf2R1 released
task1.c:52:28: Dereference of null pointer ptr: *ptr
  A possibly null pointer is dereferenced.  Value is either the result of a
  function which may return null (in which case, code should check it is not
  null), or a global, parameter or structure field declared with the null
  qualifier. (Use -nullderef to inhibit warning)
   task1.c:32:14: Storage ptr becomes null
task1.c:54:3: Assignment of void * to int: *ptr = malloc(sizeof(int))
  Types are incompatible. (Use -type to inhibit warning)
task1.c:55:3: Fresh storage *ptr (type int) not released before assignment:
                *ptr = 10
  A memory leak has been detected. Storage allocated locally is not released
  before the last reference to it is lost. (Use -mustfreefresh to inhibit
  warning)
   task1.c:54:3: Fresh storage *ptr created
task1.c:59:35: Variable ptr used after being released
   task1.c:56:8: Storage ptr released

Finished checking --- 10 code warnings
```

- Line 19 & 34 - Return value of snprintf() and gets() are ignored. Tells us to use fgets() as well because it is safer due to bounds checking.

- Line 43 - malloc is called to allocate memory for buf1R2 on line 42. If the system is out of memory, malloc returns NULL. Because we didn't check if buf1R2 == NULL before passing it to strncpy on line 43, splint warns that we might be passing a null pointer
- Line 45 - the memory was released on line 28, so freeing it again on line 45 is operating on "dead storage.", same thing as our ccpcheck and splint.
- Line 55 - On line 54, memory is allocated. On line 55, *ptr = 10; attempts to overwrite that location. Splint sees this as a potential memory leak because the newly allocated "fresh storage" hasn't been properly managed or freed.
- Line 59 - memory pointed to by ptr is freed on line 56 using free(ptr). However, on line 59, the code tries to print the value at that address:

```
printf("Use after free: %d\n", *ptr);
```

- Accessing memory after it has been returned to the system can lead to crashes or allow attackers to execute arbitrary code if they reallocate and control that memory block.

4. Clang Static Analyzer

```
colby@Colby-PC:/mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab10$ scan-build gcc task1.c -o task1
scan-build: Using '/usr/lib/llvm-18/bin/clang' for static analysis
task1.c: In function 'main':
task1.c:22:10: warning: format not a string literal and no format arguments [-Wformat-security]
   22 |   printf(buffer);
      |          ^~~~~~
task1.c:34:3: warning: implicit declaration of function 'gets'; did you mean 'fgets'? [-Wimplicit-function-declaration]
   34 |   gets(buffer);
      |   ^~~~
      |   fgets
task1.c:54:8: warning: assignment to 'int' from 'void *' makes integer from pointer without a cast [-Wint-conversion]
   54 |   *ptr = malloc(sizeof(int));
      |        ^
task1.c:49:3: warning: '__builtin_memcpy' writing 36 bytes into a region of size 10 overflows the destination [-Wstringop-overflow=]
   49 |   strcpy(mem_buffer, "Welcome, how many errors are there?");
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
task1.c:48:8: note: destination object 'mem_buffer' of size 10
   48 |   char mem_buffer[10];
      |        ^~~~~~~~~~
/usr/bin/ld: /tmp/ccsJ8vA0.o: in function `main':
task1.c:(.text+0xfa): warning: the `gets' function is dangerous and should not be used.
task1.c:34:3: error: call to undeclared function 'gets'; ISO C99 and later do not support implicit function declarations [-Wimplicit-function-declaration]
   34 |   gets(buffer);
      |   ^
task1.c:54:8: error: incompatible pointer to integer conversion assigning to 'int' from 'void *' [-Wint-conversion]
   54 |   *ptr = malloc(sizeof(int));
      |        ^ ~~~~~~~~~~~~~~~~~~
2 errors generated.
scan-build: Analysis run complete.
scan-build: 0 bugs found.
scan-build: The analyzer encountered problems on some source files.
scan-build: Preprocessed versions of these sources were deposited in '/tmp/scan-build-2026-03-15-132120-2838-1/failures'.
scan-build: Please consider submitting a bug report using these files:
scan-build:   http://clang-analyzer.llvm.org/filing_bugs.html
colby@Colby-PC:/mnt/c/Users/wcrut/OneDrive/Documents/GitHub/SecureCoding/Labs/Lab10$
```

- Line 22: Format string vulnerability at printf(buffer);
- Line 34: Unsafe input function using gets(buffer);
- Line 49: Stack buffer overflow from copying a long string into char mem_buffer[10]
- Line 54: Invalid pointer/integer assignment at *ptr = malloc(sizeof(int));

Comparative Analysis

Overall, the four tools overlap on several major issues, but each emphasizes different classes of problems. cppcheck was effective at identifying memory safety errors such as out-of-bounds access, null-pointer dereference, and double free. Flawfinder focused more on dangerous library functions and insecure coding patterns such as gets, strcpy, and uncontrolled format strings. Splint provided more semantic and flow-based warnings, including ignored return values, potential null-pointer use, dead storage, and use-after-free behavior. Clang Static Analyzer additionally exposed compile-time and type-related errors, showing that some defects prevented deeper analysis from completing.

Task 2

My code:

```python
import re

def analyze_c_file(filepath):
    # Dictionary mapping regex patterns to specific warning messages
    vulnerabilities = {
        r'\bgets\s*\(': "CRITICAL: Deprecated function 'gets()' found. Inherently vulnerable to buffer overflows.",
        r'\bstrcpy\s*\(': "WARNING: Unsafe function 'strcpy()' found. Does not check bounds.",
        r'\bsprintf\s*\(': "WARNING: Unsafe function 'sprintf()' found. Can cause buffer overflows.",
        r'\bvsprintf\s*\(': "WARNING: Unsafe function 'vsprintf()' found. Can cause buffer overflows.",
        r'\bitoa\s*\(': "WARNING: Non-standard function 'itoa()' found. Use snprintf() instead.",
        r'\bscanf\s*\(\s*\"%s\"': "CRITICAL: Unbounded 'scanf()' format string found. Allows buffer overflows."
    }

    print(f"--- Scanning '{filepath}' for vulnerabilities ---\n")

    try:
        with open(filepath, 'r') as file:
            # Read file line by line, tracking the line number
            for line_num, line in enumerate(file, 1):
                # Check the current line against all our regex patterns
                for pattern, message in vulnerabilities.items():
                    if re.search(pattern, line):
                        # Output the required format: Message, Line Number, Code
                        print(f"Issue: {message}")
                        print(f"Line Number: {line_num}")
                        print(f"Code snippet: {line.strip()}\n")
                        print("-" * 50)

    except FileNotFoundError:
        print(f"Error: Could not find the file '{filepath}'.")

if __name__ == "__main__":
    # Run the analyzer on task2.c
    analyze_c_file('task2.c')
```

Figure 1: My scan.py

For Task 2, I wrote a Python script that uses the re library to scan our C code for vulnerable or deprecated functions, basically acting like a custom, lightweight version of Flawfinder. To do this, I set up a dictionary that maps specific regex patterns—like `bgets\s*\(` or unbounded scanfs—to custom warning messages. The script opens task2.c and reads it line by line using enumerate() so I can keep track of the exact line numbers. As it loops through the file, it checks each line against my dictionary of dangerous functions using re.search(), and whenever it

gets a hit, it prints out the specific warning message, the line number, and the stripped line of code so the user knows exactly where the vulnerability is

My output:

```
colby@Colby-PC:Lab10$ python3 scan.py task2.c
--- Scanning 'task2.c' for vulnerabilities ---

Issue: CRITICAL: Deprecated function 'gets()' found. Inherently vulnerable to
buffer overflows.
Line Number: 85
Code snippet: gets(current->id);

--------------------------------------------------
Issue: WARNING: Unsafe function 'sprintf()' found. Can cause buffer overflows.
Line Number: 169
Code snippet: sprintf(index, "Insert: %s", "Index Out of Bounds");

--------------------------------------------------
Issue: WARNING: Non-standard function 'itoa()' found. Use snprintf() instead.
Line Number: 218
Code snippet: itoa(-123456789, current->id, 10);

--------------------------------------------------
Issue: WARNING: Unsafe function 'vsprintf()' found. Can cause buffer overflows.
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

--------------------------------------------------
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);
Line Number: 234
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

--------------------------------------------------
Issue: CRITICAL: Unbounded 'scanf()' format string found. Allows buffer overflows.
Line Number: 269
Code snippet: scanf("%s", input);
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

--------------------------------------------------
Issue: CRITICAL: Unbounded 'scanf()' format string found. Allows buffer overflows.
```

```
Line Number: 269
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

------------------------------------------------
Issue: CRITICAL: Unbounded 'scanf()' format string found. Allows buffer overflows.
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

------------------------------------------------
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);
Line Number: 234
Line Number: 234
Line Number: 234
Line Number: 234
Code snippet: vsprintf(msg_buf, fmt, args);

------------------------------------------------
Issue: CRITICAL: Unbounded 'scanf()' format string found. Allows buffer overflows.
Line Number: 269
Code snippet: scanf("%s", input);
```